ROUND 1 TECHNICAL PROPOSAL BLUEPRINT

Handover Ledger

An Evidence-Linked, Role-Scoped Knowledge Reconstruction Platform for Team Continuity

Team: Padminish Bakshi & Panav Khungar

Target Scope: Small Teams, Student Clubs & Research Labs

Status: Round 1 Submission Blueprint

Core Originality Hook: Why Handover Ledger is Fundamentally Different

While typical products rely on "LLM chat summaries" that hallucinate and obscure contradictions, **Handover Ledger** re-engineers team succession around verified evidence and access boundaries:

1. **Atomic Claims over Loose Summaries:** Knowledge is broken into discrete units (decisions, owners, blockers, risks, open questions) with click-through citations, confidence scores, and status flags (`Verified` , `Inferred` , `Conflicting` , `Missing`).
2. **Departing-Person Interview Loop:** The engine automatically pinpoints undocumented infrastructure gaps (e.g., *"Cron job owned by Rahul with no runbook"*) and asks the departing member 5–10 targeted prompt questions to generate high-trust claims.
3. **Permission-First Data Isolation:** Role-based access filtering occurs before inference and during presentation, guaranteeing confidential context never leaks to unapproved members.
4. **Actionable Continuation Blueprint:** Delivers an operative Day-1 / Week-1 / Month-1 execution plan with formal successor acknowledgement, not an unread dump.

1. PROBLEM UNDERSTANDING & SCOPING

Project handover failure causes critical knowledge attrition across engineering and research groups. The root problems stem from:

- **Tacit Knowledge Vanishing:** Technical decisions, discarded architectures, unrecorded workarounds, half-finished prototypes, and informal vendor contacts reside only in the leaver's head.
- **Static Documentation Rot:** Wikis, READMEs, and Notion pages are written once, become stale rapidly, and fail to record rationale, doubts, and ongoing risks.
- **Zero-Trust AI Summaries:** Standard LLM summarizers hallucinate, smooth over conflicting accounts, blur exact source grounding, and ignore data security classifications.
- **Explicit Project Boundaries:** We explicitly target small-team project handovers (student organizations, lab groups, seed-stage startups) operating on GitHub, Docs/Markdown, tasks, and chat exports. We avoid enterprise ERP ingestion to ensure extreme fidelity, airtight citation grounding, and feasible delivery.

2. TARGET USERS & SCENARIOS

STAKEHOLDER	CORE PAIN POINT	HANDOVER LEDGER VALUE
Departing Member	Overwhelmed by exit documentation requests; leaves gaps accidentally.	Automated extraction + 5-minute targeted interview loop to capture high-value tacit context.
Incoming Successor	Overwhelmed by sprawling repositories; unassigned bugs, lost contacts.	Prioritized Day-1 / Week-1 / Month-1 continuation plan with evidence-linked ownership.
Project Lead / Reviewer	Uncertain what is true, missing, or confidential before handoff.	Review console to verify claims, inspect conflicts, and approve scoped release packages.
Org Lead / Administrator	Risk of accidental credential/privacy leak and missing auditability.	Role-scoped view filters with immutable receipt logging and audit trails.

Primary Scenarios: Student graduation, research assistant handover, intern completion, unexpected resignations, and project rotation.

3. PROPOSED SOLUTION ARCHITECTURE

A web application that ingests approved repositories and documents, parses atomic claims grounded in precise text spans, executes an interview loop to capture missing institutional knowledge, allows project leads to review and edit claims, and publishes scoped handover reports requiring successor acknowledgement.

4. 5-STEP PRODUCT WORKFLOW

STEP 1 **Connect & Ingest**

Upload GitHub repositories, task exports (CSV/JSON), meeting notes, and chat exports. Each source is tagged with an organizational sensitivity label and allowed roles at upload.

STEP 2 **Reconstruct & Chunk**

Sources are indexed and parsed into structured claims: architectural decisions, component owners, active blockers, recurring issues, external contacts, and a chronological milestone timeline.

STEP 3 **Ground, Verify & Query**

Every claim is matched against source citations and assigned a verification tier (`Verified` , `Inferred` , `Conflicting`). Structural gaps trigger a tailored 5–10 question interview for the departing member.

STEP 4 **Review Console**

The leaver reviews questions; the team lead inspects extracted claims, edits inaccuracies, resolves conflicts, and signs off. Every modification is recorded in an immutable version log.

STEP 5 **Role-Scoped Handover & Signoff**

The successor views a scoped interface (e.g., Core Dev vs. Project Coordinator) with restricted context masked out, a clear 30-day continuation checklist, and a required formal acknowledgement of receipt.

5. CORE MVP FEATURES

- **Multi-Source Parsing:** Markdown, GitHub commits/PRs, exported task boards, and raw chat logs.
- **Evidence-Linked Claim Engine:** Side-by-side click-through citation to exact source lines and documents.
- **Confidence & Conflict Badging:** Explicit distinction between documented facts and model inferences.
- **Gap-Driven Leaver Interview:** Dynamic prompt generator that asks the questions documentation missed.
- **Lead Review & Edit Console:** Single-click approve, modify, reject, and add claims with change tracking.
- **Role-Filtered Views:** Redaction masks sensitive items from general or junior members automatically.
- **30-Day Continuation Roadmap:** Prioritized schedule defining Day-1 setups, Week-1 tasks, and Month-1 milestones.
- **Cryptographic Acknowledgement & Audit Log:** Successor digital receipt confirmation for team governance.

6. TECHNOLOGY STACK & IMPLEMENTATION STRATEGY

LAYER	TECHNOLOGY	ARCHITECTURAL RATIONALE
Frontend	Next.js 14, Tailwind CSS, TypeScript	Server-rendered responsive UI, zero bloat, instant inspection of side-by-side claims and citations.
Backend API	FastAPI (Python 3.11)	High-performance async parsing, native compatibility with ML/LLM pipelines and JSON schema validation.
Database & Vectors	PostgreSQL with pgvector extension	Single relational store combining user authentication, RBAC rules, raw claims, and vector embeddings.
LLM Inference	Claude API (Sonnet) with Structured JSON Output	Superior adherence to strict JSON schemas; eliminates malformed outputs; enforces atomic claim shapes.
Auth & Permissions	Supabase Auth / NextAuth with Row-Level Security	Strict database-level enforcement guaranteeing zero unauthorized retrieval during ingestion or render.
Deployment	Vercel (UI) + Render / Railway (API & DB)	Zero-friction CI/CD pipeline with independent scaling of frontend and ingestion workers.

The Hardest Technical Problem: Verifiable Grounding & Zero Leakage

Preventing hallucinated claims and role leaks represents our primary architectural hurdle. We solve this through a deterministic four-tier defense:

1. **Pre-Retrieval Role Guarding:** The database filters documents matching the user's role before vector search or LLM context construction begins.
2. **Span Quoting Mandate:** The model must return the precise verbatim span alongside source identifiers.
3. **Automated String Verification Pass:** Backend algorithms check that the quoted span exists verbatim in the raw source. Claims failing this check are rejected automatically.
4. **Deterministic Conflict Surfacing:** When multiple sources or the leaver contradict previous records, both claims are preserved and flagged with conflicting badges rather than silently merged.

7. TEAM RESPONSIBILITIES & WORK ALLOCATION

TEAM MEMBER	ROUND 1 ARCHITECTURE & RESEARCH	ROUNDS 2 & 3 ENGINEERING DELIVERABLES
Padminish Bakshi *Backend & AI Systems*	Evidence data modeling, source ingestion architecture, prompt design for claim extraction and conflict analysis.	FastAPI backend, pgvector integration, Claude JSON pipeline, string-matching citation verifier, and GitHub REST API integration.
Panav Khungar *Product & Frontend Systems*	UX workflow diagrams, user interview synthesis, role-permission matrix design, and continuation roadmap UI specs.	Next.js dashboard, side-by-side citation viewer, review & edit console, role-based redaction renderer, and acknowledgement flow.
Joint Ownership	Synthetic test suite design, problem validation survey, end-to-end integration testing, documentation, and video demonstration.	

8. MILESTONE-DRIVEN DEVELOPMENT PLAN

MILESTONE	CORE OBJECTIVES	KEY OUTPUT DELIVERABLES
Round 1 *Blueprint & Research*	Formalize problem scope, interview lead developers, define schema schemas, and construct synthetic benchmark data.	Architectural blueprint, finalized data schemas, and high-fidelity prototype layouts.
Round 2 *Functional MVP*	Complete upload pipeline, claim extraction with citation linking, status badges, review console, and single-role export.	Deployable web application demonstrating end-to-end extraction from a live repository.
Round 3 *Hardened System*	Implement leaver interview loop, multi-role redaction, automated conflict detection, and formal successor acknowledgement.	Production-ready build, test coverage, accessible UI, and recorded video walkthrough.

9. EXPECTED IMPACT & SUCCESS METRICS

- **Onboarding Velocity:** 50% decrease in time-to-first-contribution for incoming maintainers.
- **Zero Orphaned Components:** 100% of discovered scripts, cron jobs, and credentials have designated successor owners.
- **Auditability:** 100% of presented claims link to verifiable source passages or direct leaver attestations.
- **Confidentiality Protection:** Absolute mathematical containment of restricted sources across filtered role views.

10. FUTURE ROADMAP & EXTENSIONS

Subsequent iterations will include direct live connectors for Slack, Microsoft Teams, Jira, and Notion; an automated "continuous living handover" engine that tracks organizational knowledge before member departure; single sign-on (SSO) integration; and an organizational "bus factor" risk monitor calculating knowledge vulnerabilities across teams.

11. PROJECT DECLARATIONS & INTEGRITY

Team Declarations: Prepared by Padminish Bakshi and Panav Khungar. All evaluation datasets, repositories, and user testimonials utilized for demonstration and testing will be fully synthetic or sanitized to respect privacy guidelines. AI assistance was utilized strictly for code ideation and structural formatting; all core solution designs, verification logic, and architectures represent original student team work.